



NLP: Text Preprocessing

Contents

- Conda Setup
- NLP Overview
- Text Preprocessing
- Vectorization
- Sentiment Analysis
- Text Classification
- Topic Modelling
- Neural Networks Review
- Deep Learning Review
- Transformers & LLMs

Conda Setup

Category	Command	Description
Environment	<code>conda env list</code>	Lists all conda environments in your system
	<code>conda create --name <i>test_env</i></code>	Creates a new environment called <i>test_env</i>
	<code>conda activate <i>test_env</i></code>	Activates the <i>test_env</i> environment
	<code>conda deactivate</code>	Deactivates the current environment and returns to the base environment
Package	<code>conda list</code>	Lists installed packages in the active environment
	<code>conda install</code>	Installs specified packages into the active environment
YML	<code>conda env export > <i>test_env.yml</i></code>	Exports package names and versions in the active environment into a .yml file
	<code>conda env create -f <i>test_env.yml</i></code>	Create a new environment from a .yml file

NLP Overview

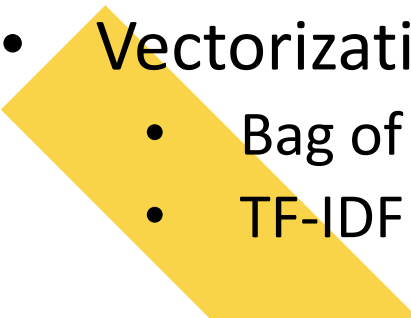
- AI
 - CV
 - ML
 - NLP: Make computers work with human language. Use computers to analyze, understand, manipulate written or spoken examples of human language. Eg: Sentiment Analysis, Text Summarization, Question-Answering, etc.

NLP Overview: History

- Early NLP
 - Rule Based Techniques
- Traditional NLP
 - Statistical, Hand-crafted, Classical ML
- Modern NLP
 - Deep Learning: Recurrent Networks & Transformers

Text Preprocessing



- Cleaning & Normalization
 - Lowercasing
 - Remove punctuations, special characters
 - Tokenization
 - Stemming/Lemmatization
 - Remove Stop Words
 - PoS tagging
 - Vectorization
 - Bag of Words
 - TF-IDF
- 

DTM or Bag of Words

text_clean

```
0      life lemon lemonade
1      lemon maven market
2      dozen lemon gallon lemonade
3      lemon lemon lemon lemon lemon lemon
4      s market lemon sale today
5      maven market eureka lemon lemon
6      palmer lemonade half ice tea
7      ice tea favorite
Name: sentence, dtype: object
```



	ice	lemon	lemonade	market	maven	tea
0	0	1	1	0	0	0
1	0	1	0	1	1	0
2	0	1	1	0	0	0
3	0	6	0	0	0	0
4	0	1	0	1	0	0
5	0	2	0	1	1	0
6	1	0	1	0	0	1
7	1	0	0	0	0	1

TF-IDF

Term Frequency

Problem it solves:

High counts can dominate, especially for high frequency words or long documents

Solution:

Normalize the counts so they're all on the same scale

Inverse Document Frequency

Problem it solves:

Each word is treated equally, even when some might be more important

Solution:

Assign more weight to rare words than to common words

$$TF-IDF = \frac{\text{Term count in document}}{\text{Total terms in document}} * \log \left(\frac{\text{Total documents} + 1}{\text{Documents with the term} + 1} \right)$$

ML Overview

Supervised Learning

Using historical data to predict the future



What will house prices look like for the next 12 months?



How can I flag suspicious emails as spam?

Unsupervised Learning

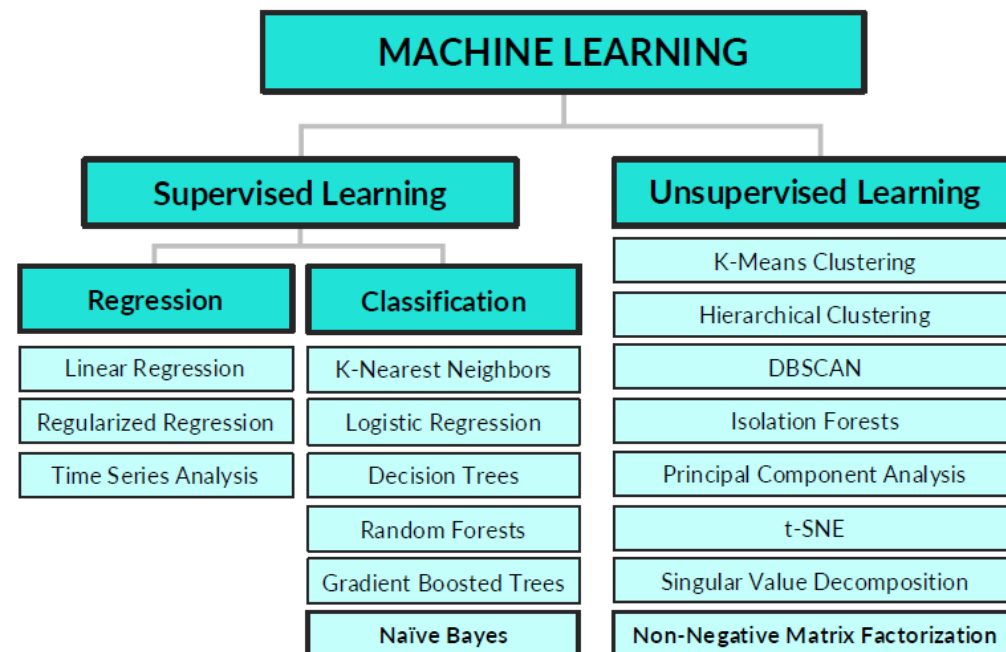
Finding patterns and relationships in data



How can I segment my customers?



What hidden themes are in these product reviews?



Popular NLP Tasks with Traditional Methods



Sentiment Analysis

Identifying the positivity or negativity of text

- **Technique:** Rules-based
- **Library:** VADER
- **Input format:** raw text



Text Classification

Classifying text as one label or another

- **Technique:** Supervised learning (i.e., Naïve Bayes)
- **Library:** scikit-learn
- **Input format:** CV / TF-IDF



Topic Modeling

Finding themes within a corpus of text

- **Technique:** Unsupervised learning (i.e., NMF)
- **Library:** scikit-learn
- **Input format:** CV / TF-IDF

Traditional vs Modern Techniques

What is my NLP goal?

Sentiment Analysis

Text Classification

Topic Modeling

Text Generation

Machine Translation

Question Answering

These can be done with
traditional techniques

How much data do I have?

Small to medium data
($< 100k$ rows)

Big data ($> 1M$ rows)

Try traditional
techniques first

Consider modern
techniques

These cannot be done with traditional
techniques, so **use modern techniques**

Sentiment Analysis

- Can be done based on Rules or Classification Techniques or through Modern techniques
- Rule based: VADER (for informal language typically seen on social media)
 - Dictionary of words and phrases have an assigned sentiment.
 - Punctuation, capitalization, dampening words modify intensity of sentiment.
 - Individual sentiments are combined to create an overall sentiment score
 - Used for customer feedback analysis, social media analysis.
 - Alternatives: TextBlob, SentiWordNet

Text Classification

- Requires labelled data
- Use Vectorized Text to build and infer from Classifiers
- All supervised ml algorithms can be used
- Use models such as Naïve Bayes, Logistic Regression for less than 10K records. For large datasets use GBTs and Neural Networks
- Applications: Support Ticket Identification, Spam Classification

Naïve Bayes

EXAMPLE

If an email contains the word "ASAP", how likely is it going to be spam?

Distribution of 1,000 emails:

		SPAM	
		1	0
ASAP	1	50	10
	0	200	740



$$\frac{P(\text{ASAP}|\text{Spam}) * P(\text{Spam})}{P(\text{ASAP})} = \frac{\frac{50}{250} * \frac{250}{1000}}{\frac{60}{1000}} = \frac{0.2 * 0.25}{0.06} \approx 0.83$$

Naïve Bayes

EXAMPLE

If an email contains the word "ASAP" and the "\$" symbol, how likely is it going to be spam?

Distribution of 1,000 emails:

		SPAM	
		1	0
ASAP	1	50	10
	0	200	740
\$	1	80	30
	0	170	720



$$\frac{P(\text{ASAP}|\text{Spam}) * P(\$|\text{Spam}) * P(\text{Spam})}{P(\text{ASAP}, \$)} \approx 0.94$$



These probabilities are all calculated automatically by the model!



There's a 94% chance an email is spam if it contains "ASAP" and "\$"

ASAP and \$: 17

Topic Modelling

	Topic 1	Topic 2
<i>"I like lemons and limes."</i>	100%	0%
<i>"Puppies and kittens are so cute."</i>	0%	100%
<i>"I'm making cat-shaped cookies."</i>	50%	50%
<i>"My dog loves apples and blueberries."</i>	67%	33%



What are topics 1 and 2?

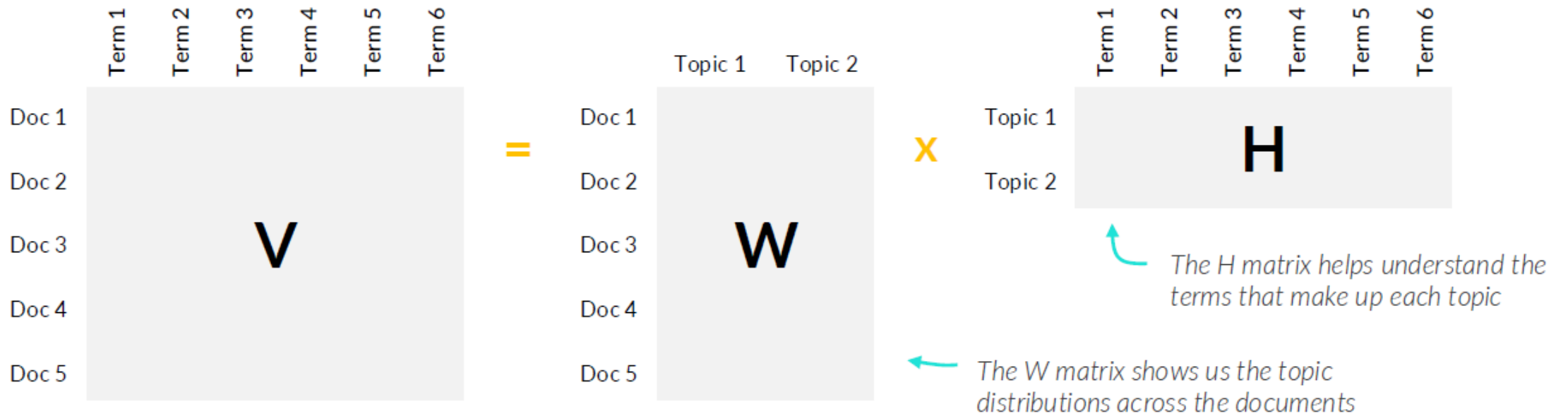
- Topic 1: lemons, limes, cookies, apples, blueberries
- Topic 2: puppies, kittens, cat, dog

Food

Animals

Non-Negative Matrix Factorization

- Decomposes the document term matrix into two other matrices:
 - A document topic matrix (W) that shows how much each topic appears in each document
 - A topic-term matrix (H) shows how relevant each word is to each topic



Traditional MLP vs Modern MLP



Traditional NLP

Data:

- Small to medium data sets

Techniques:

- Rules-based
- Supervised learning (Naïve Bayes)
- Unsupervised learning (NMF)

Applications:

- Sentiment analysis
- Text classification
- Topic modeling



Modern NLP

Data:

- Small to large data sets

Techniques:

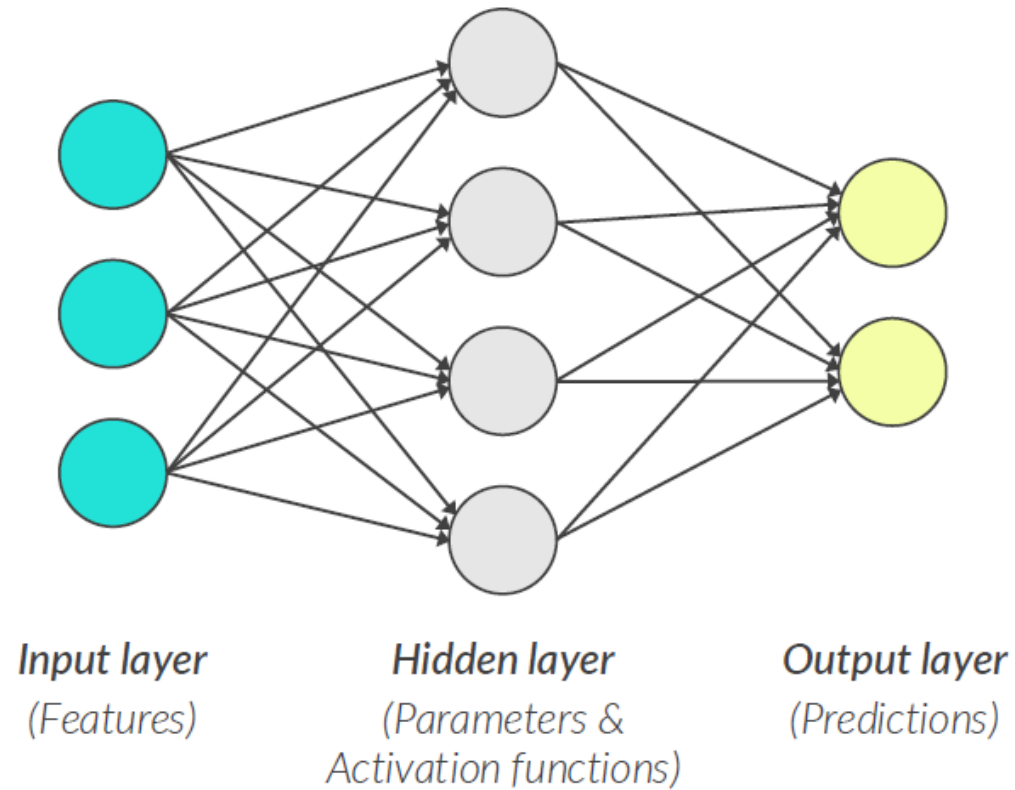
- Transformers-based LLMs (BERT, GPT, LLaMA, T5, BART)

To understand transformer-based models, we'll start with the basics: neural networks

Applications:

- Traditional NLP applications
- Text summarization
- Text generation

Neural Networks Review

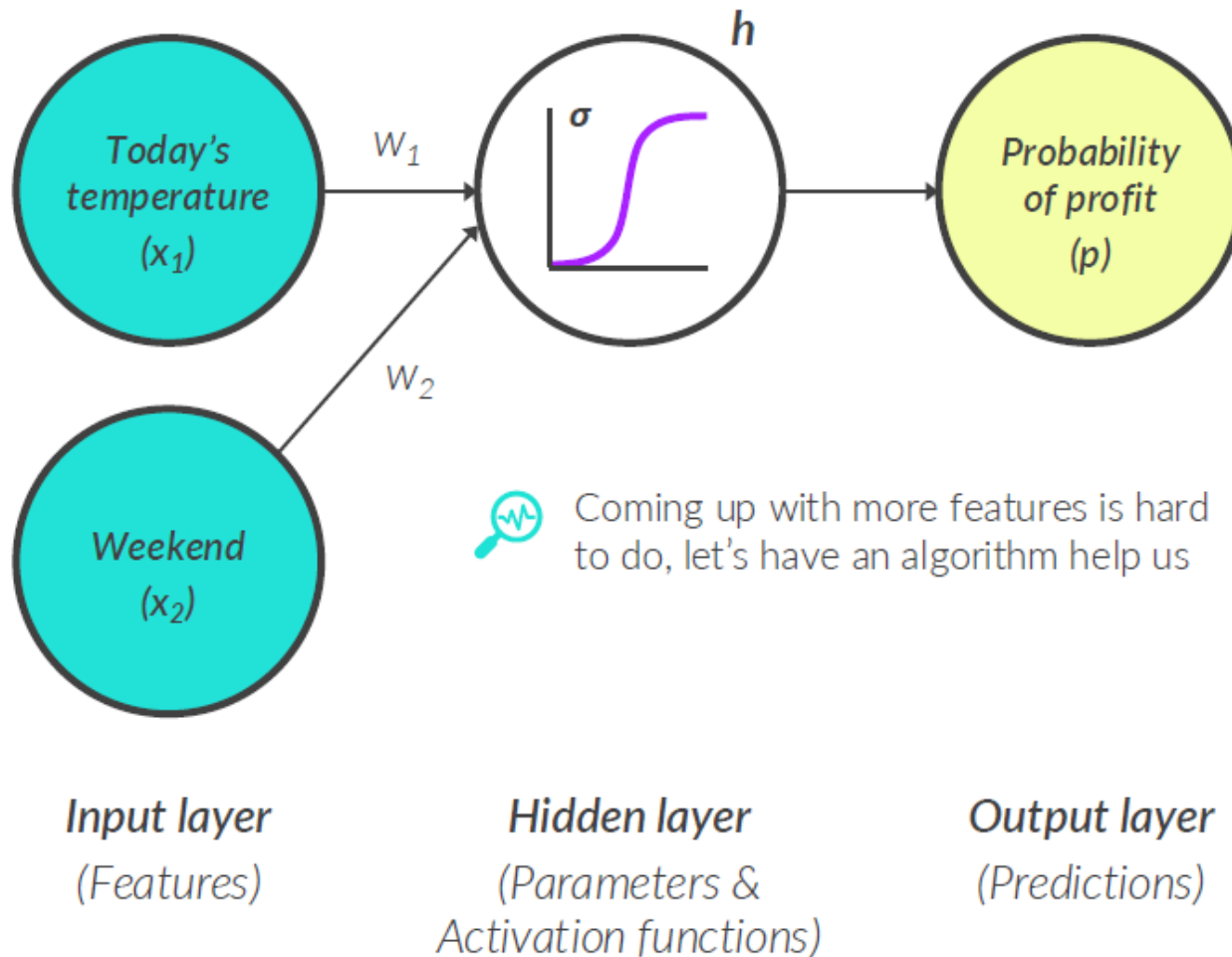


Neural Networks Review

EXAMPLE

Based on today's temperature, will my lemonade stand be profitable today?

These **w** and **b** values come from fitting a logistic regression model in scikit-learn



$$h = \sigma(w_1x_1 + w_2x_2 + b)$$

$$h = \sigma(0.25x_1 + 1x_2 - 5)$$

$$x_1 = 15, x_2 = 0 \rightarrow h = 22\%$$

$$x_1 = 15, x_2 = 1 \rightarrow h = 43\%$$

$$x_1 = 25, x_2 = 0 \rightarrow h = 77\%$$

$$x_1 = 25, x_2 = 1 \rightarrow h = 90\%$$

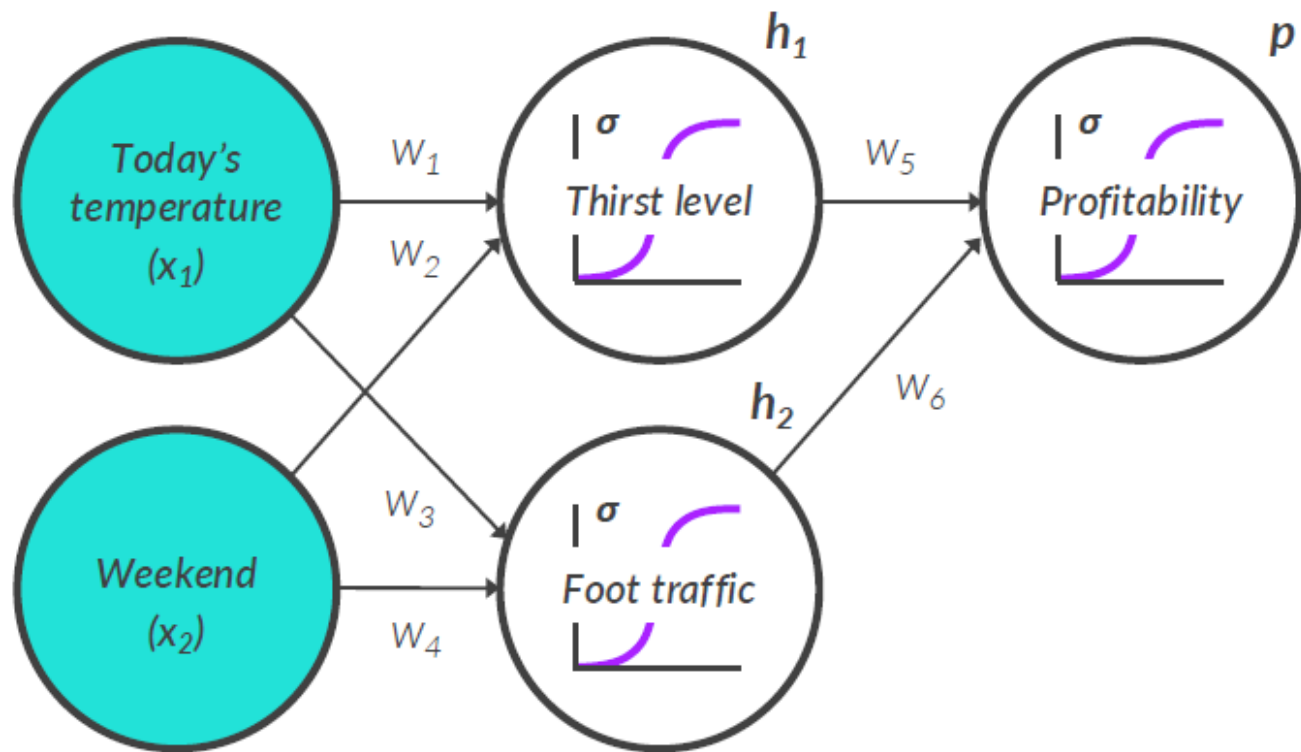
$$x_1 = 35, x_2 = 0 \rightarrow h = 97\%$$

$$x_1 = 35, x_2 = 1 \rightarrow h = 99\%$$

Neural Networks Review

EXAMPLE

Based on today's temperature, will my lemonade stand be profitable today?



Input layer
(Features)

Hidden layer
(Parameters &
Activation functions)

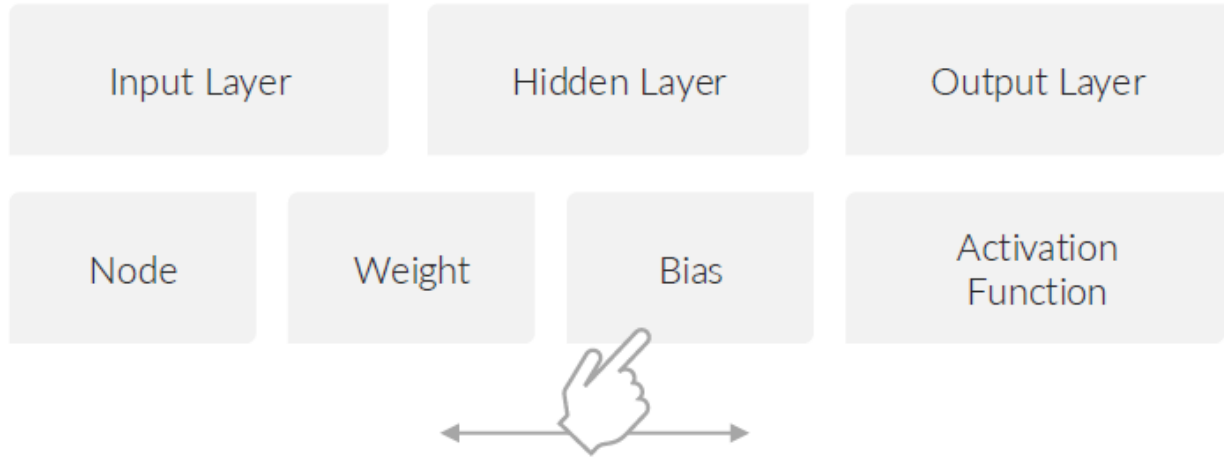
Output layer
(Predictions)



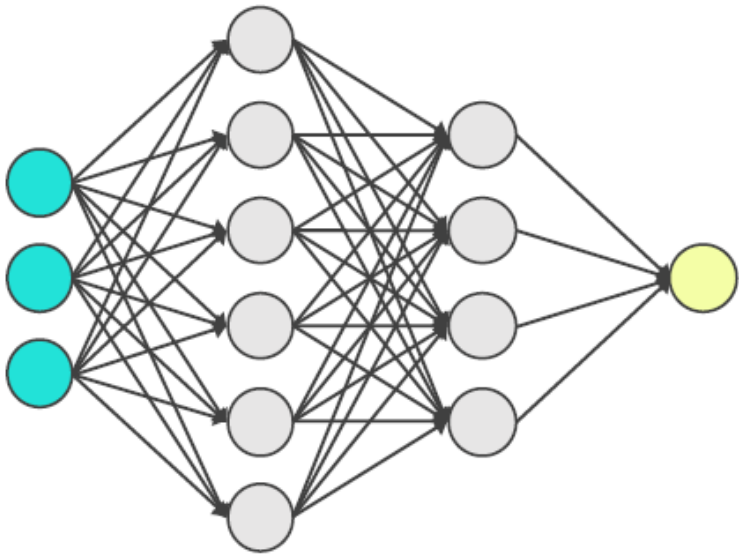
$$\begin{aligned}h_1 &= \sigma(w_1x_1 + w_2x_2 + b_1) \\h_2 &= \sigma(w_3x_1 + w_4x_2 + b_2) \\p &= \sigma(w_5h_1 + w_6h_2 + b_3)\end{aligned}$$

The outputs from the hidden layers are assigned their own weights and bias, and wrapped in a final activation function

Neural Networks Exercise



- *How many layers?*
- *How many hidden layers?*
- *How many nodes?*
- *How many weights?*
- *How many biases?*
- *How many parameters?*
- *How many activation functions?*

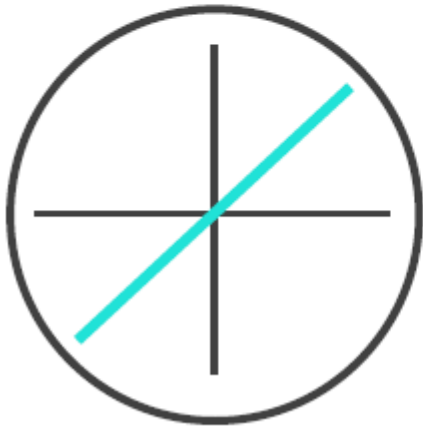


$$h_1 = \sigma(w_1x_1 + w_2x_2 + w_3x_3 + b_1)$$

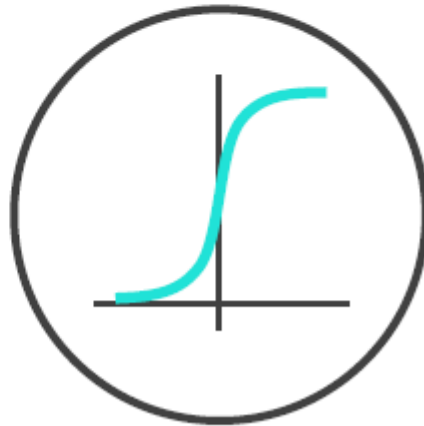
Neural Networks Review

```
from sklearn.neural_network import MLPClassifier  
  
nn = MLPClassifier(hidden_layer_sizes=(100,), activation='relu')
```

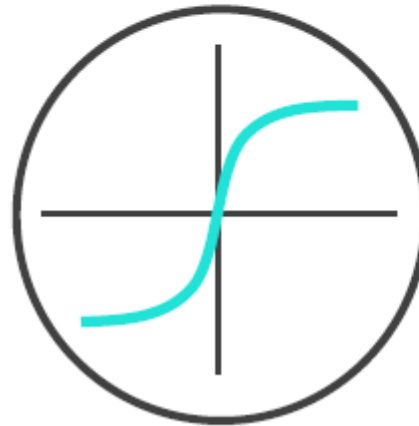
Identity



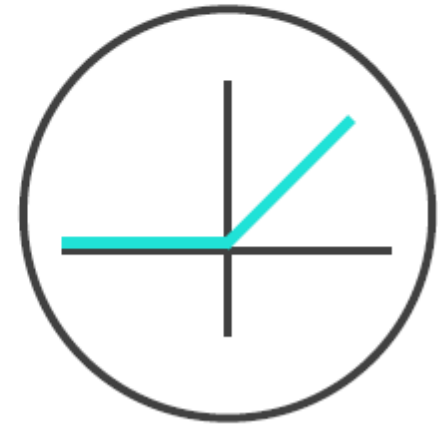
Logistic



Tanh



ReLu

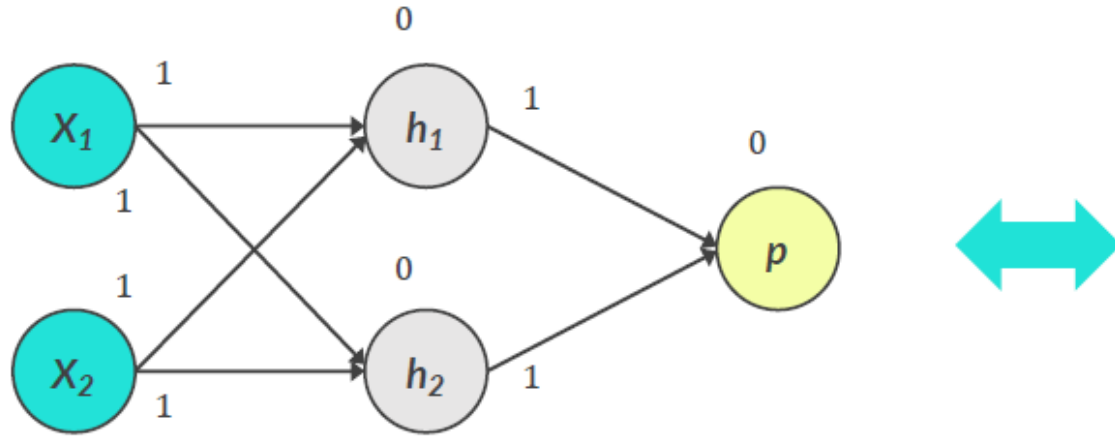


Neural Networks Review

Training a neural network means to calculate its optimal parameters

1. **Random start:** Start with an initial set of random weights & biases
2. **Forward pass:** Starting from the left, apply all calculations through the neural network to get to a final set of predicted values
3. **Calculate loss:** Compare the predicted and actual values to compute the error, or loss
4. **Update parameters:** Starting from the right, calculate how much each parameter contributed to the loss with *back propagation*, and then use *gradient descent* (a popular optimization technique) to adjust the parameters by moving them a step closer to reducing the loss
5. **Repeat:** Repeat steps 2-4 until you minimize the loss or reach an iteration limit and lock in the final model parameters (weights and biases)

Neural Networks Training



$$h_1 = \sigma(x_1 + x_2)$$

$$h_2 = \sigma(x_1 + x_2)$$

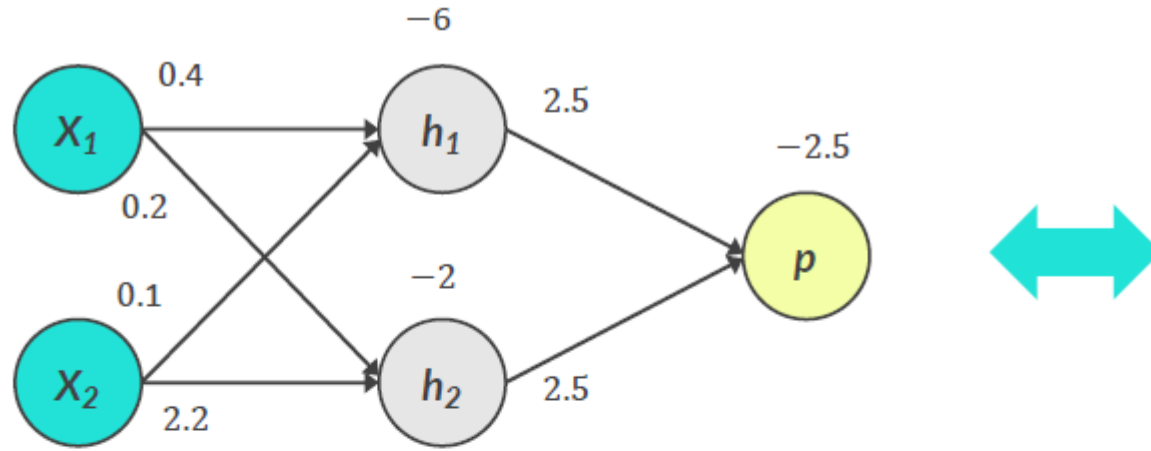
$$p = \sigma(h_1 + h_2)$$

Training data:

Temperature (x_1)	Weekend (x_2)	Profitable (y)	Prediction (p)	Error (ϵ)
14	0	0	0.88	-0.88
18	1	0	0.88	-0.88
22	0	0	0.88	-0.88
22	1	1	0.88	0.12
26	0	1	0.88	0.12
26	1	1	0.88	0.12
30	0	1	0.88	0.12
30	1	1	0.88	0.12
35	0	1	0.88	0.12
15	1	0	0.88	-0.88

LOG LOSS: **0.927**

Neural Networks Training



$$h_1 = \sigma(0.4x_1 + 0.1x_2 - 6)$$

$$h_2 = \sigma(0.2x_1 + 2.2x_2 - 2)$$

$$p = \sigma(2.5h_1 + 2.5h_2 - 2.5)$$

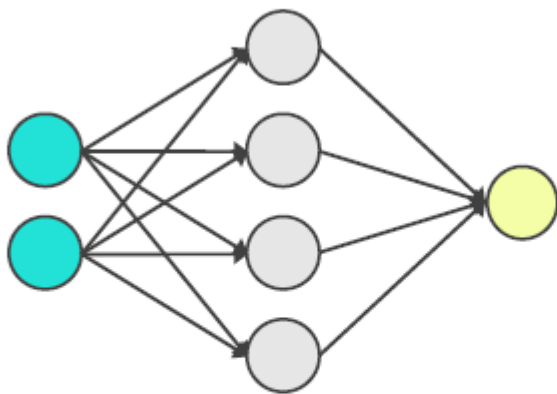
Temperature (x_1)	Weekend (x_2)	Profitable (y)	Prediction (p)	Error (ϵ)
14	0	0	0.56	-0.56
18	1	0	0.87	-0.87
22	0	0	0.9	-0.9
22	1	1	0.91	0.09
26	0	1	0.91	0.09
26	1	1	0.92	0.08
30	0	1	0.92	0.08
30	1	1	0.92	0.08
35	0	1	0.92	0.08
15	1	0	0.77	-0.77

LOG LOSS: **0.710**

This is down from 0.927!

Deep Learning

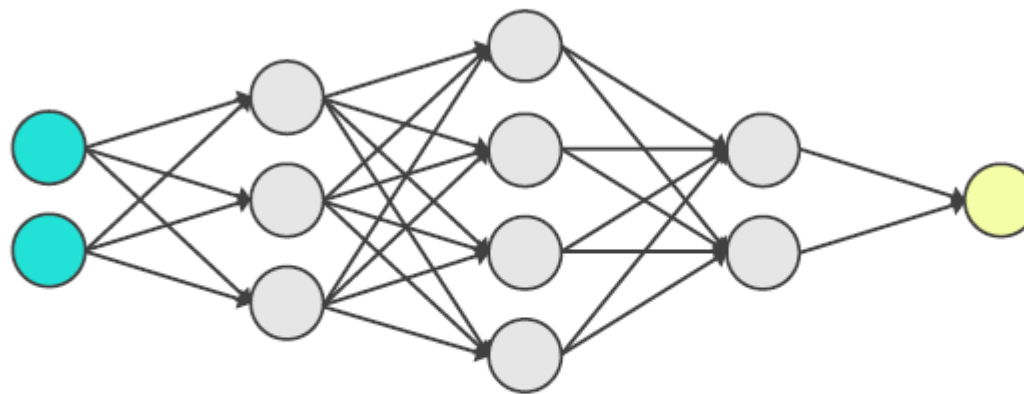
Neural Network



Sometimes used for:

- Simple classification tasks (0 or 1)
- Medium data sets (*thousands of rows*)

Deep Learning



Often used for:

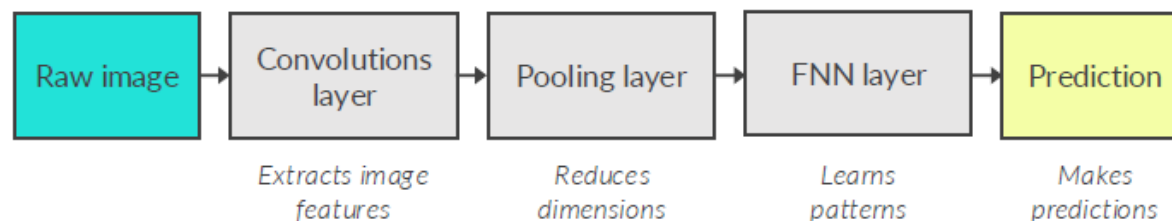
- More complex tasks (*NLP, CV, ASR, etc.*)
- Large data sets (*millions of rows*)

Deep Learning Architectures

Convolutional Neural Networks (CNNs)

Popularized in 2012

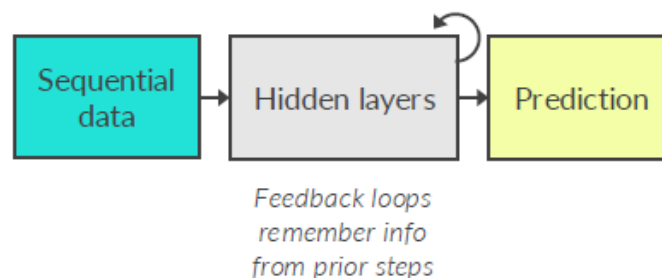
Applications:
Image-related tasks like image classification, object detection, etc.



Recurrent Neural Networks (RNNs)

Popularized in 2013

Applications:
Sequential tasks, such as NLP tasks, time series analysis, etc.

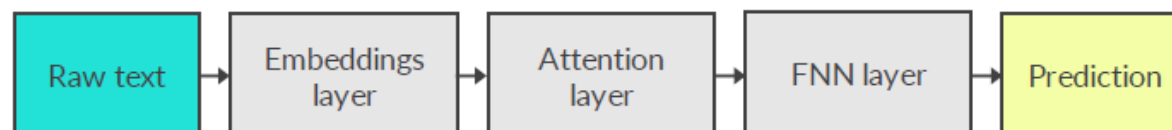


Long Short-Term Memory (LSTMs)
are an extension of RNNs that include logic to remember and forget info over time (popularized in 2015)

Transformers

Popularized in 2017

Applications:
NLP, CV, ASR tasks, and much more!



(More on these in the Transformers section up next!)

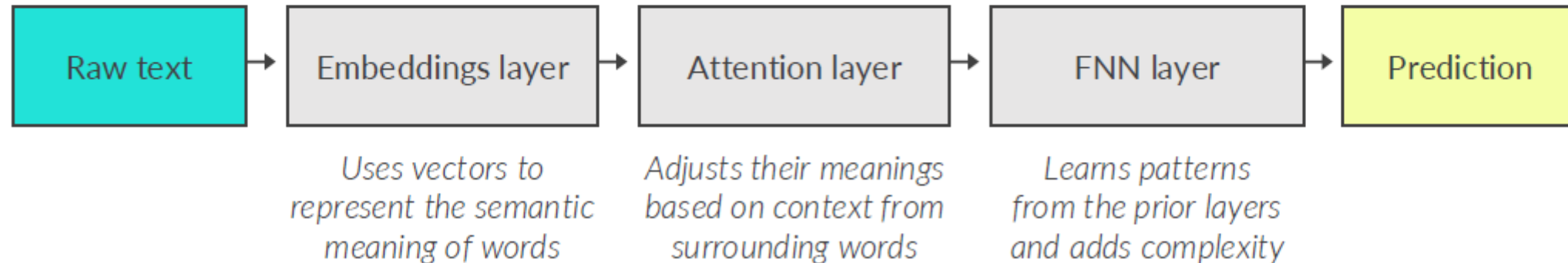
Transformers have replaced RNNs and LSTMs for NLP tasks

Transformer Architecture

The **transformer architecture** refers to the series of layers and computations that the input data passes through to produce a final result

- Along the way, the input text is gradually *transformed*, hence the name transformers

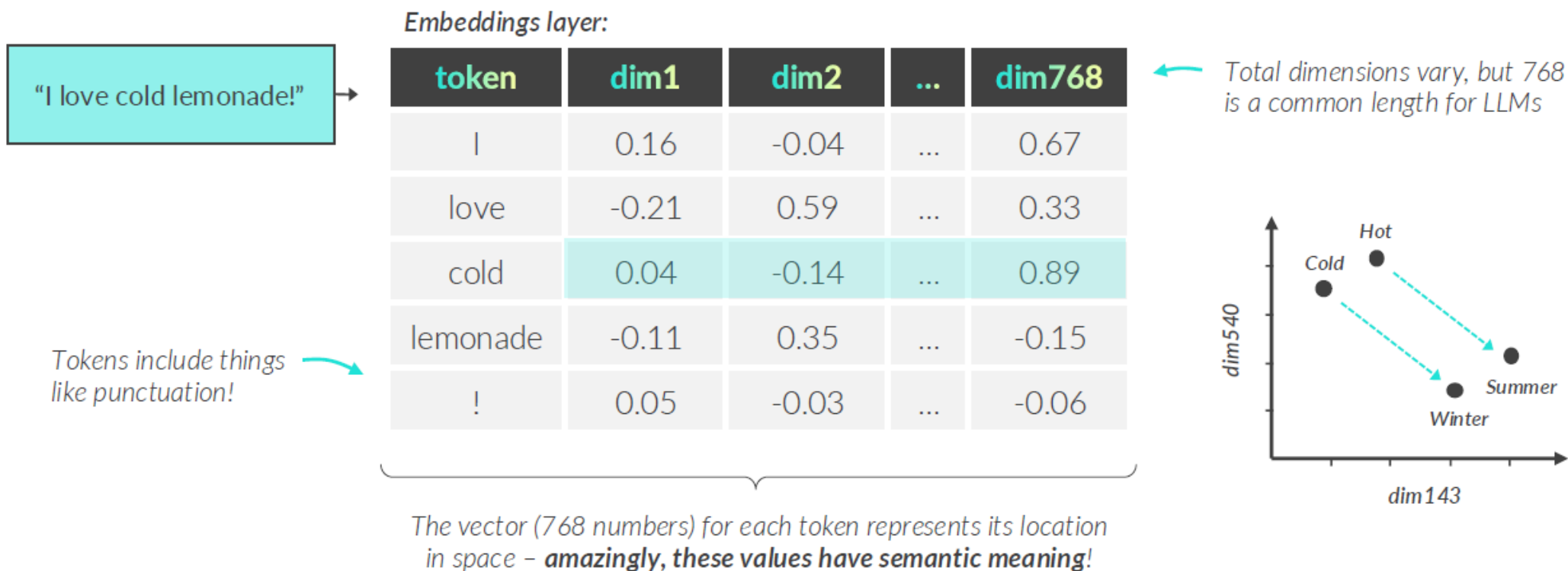
These are the **main layers** of a transformer:



Embedding Layer

The first layer of a transformer, the **embeddings** layer, converts text tokens into meaningful numeric representations

- It places each token (word) into a high-dimensional space, so words with similar meanings end up close together, and words with different meanings are farther apart



Attention Layer



“lemonade”



*This will be in an exact
location based on the
word embedding*

“I love cold lemonade!”



*This will be in a slightly
different location, since
it's specifically cold
lemonade that's loved*

“I love Beyonce's Lemonade album.”



*This will be in a very
different location, since it's
about an album, not a drink*



Attention Layer

The token here
is "lemonade"



"I love cold lemonade!"



Without the attention layer, the meaning of the word "lemonade" isn't affected by the other words in the sentence

"I love **cold** lemonade!"



With the attention layer, the word "cold" **adds context to** "lemonade"

- In technical terms: cold **attends to** lemonade
- In layman's terms: this isn't just any lemonade, it's a cold one

"I **love** cold lemonade!"



With the attention layer, the word "love" **adds context to** "lemonade"

- In technical terms, love **attends to** lemonade
- In layman's terms: this isn't just any lemonade, it's lemonade that's loved

"Lemonade" will absorb the meanings of
"cold" and "love", but not so much "I"

Attention Layer

	I	love	cold	lemonade	!
I	0.2	0.6	0	0	0.2
love	0.1	0.1	0.3	0.4	0.1
cold	0.1	0.1	0.2	0.5	0.1
lemonade	0.3	0.1	0.4	0.1	0.1
!	0.3	0.1	0.1	0	0.5

← The "love" is mostly for "lemonade",
and somewhat for "cold"

← "Cold" mostly describes "lemonade"

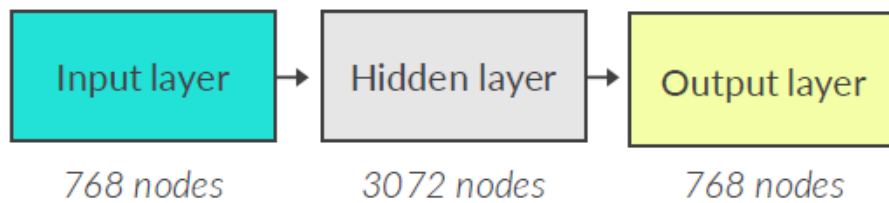
← "Lemonade" is getting "love" and is "cold"

FNN

The third layer of a transformer, the **feedforward neural network (FNN)** layer, learns patterns from the data and adds complexity to the model

- Embedding layer: words are placed in locations in space that hold some meaning
- Attention layer: the locations are adjusted based on context from surrounding words
- FNN layer: patterns in those contextual relationships are learned and captured

Typical FNN in a transformer:



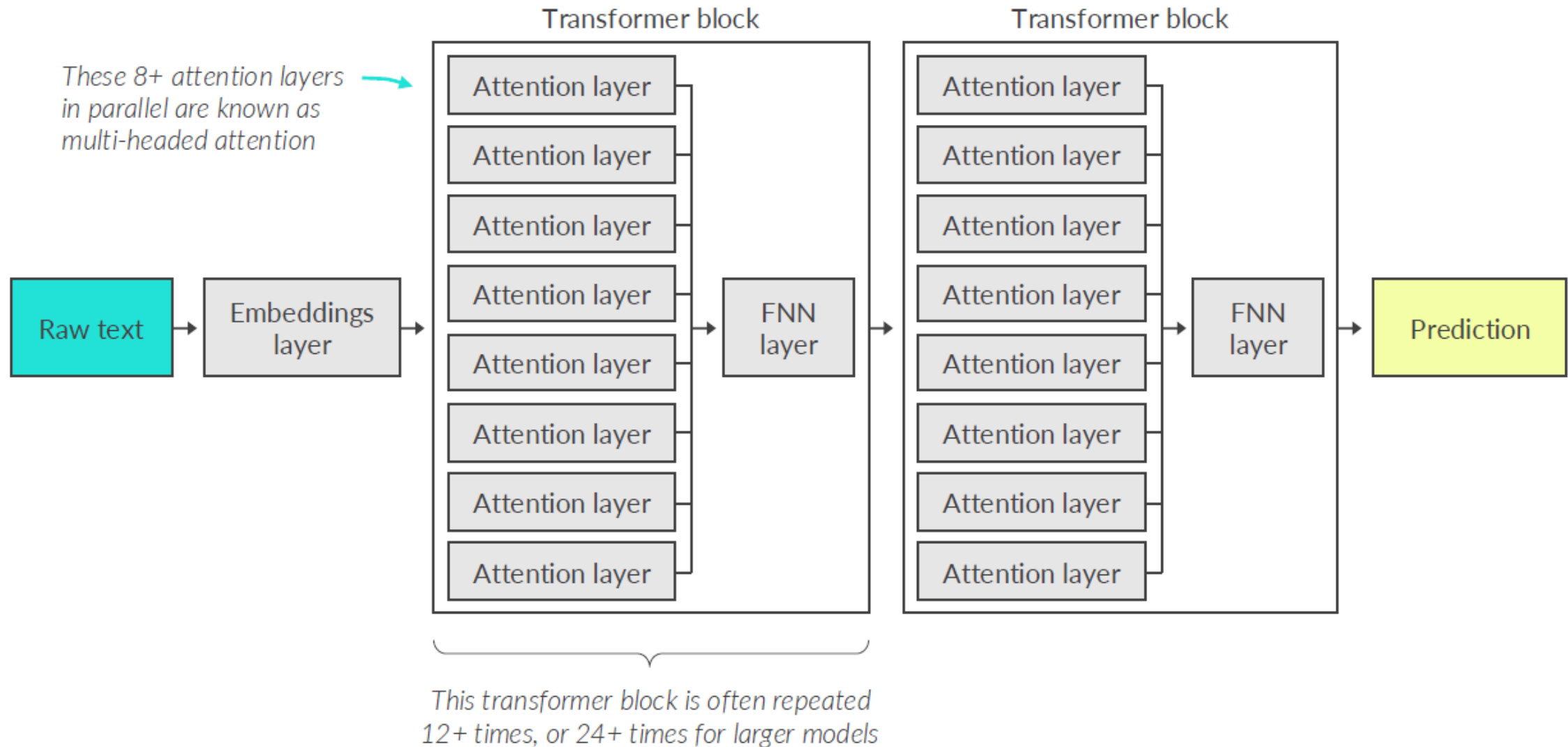
← The output is a transformed representation of the original tokens with refined meanings

One of these nodes could be capturing the idea of a noun-adjective relationship, another could be capturing the idea of love, etc.



This is just a **widely-accepted theory** or interpretation of how this works to make it more understandable – the actual workings are more abstract and less interpretable

Common Transformer



Encoder Only Model

Encoder-Only Models

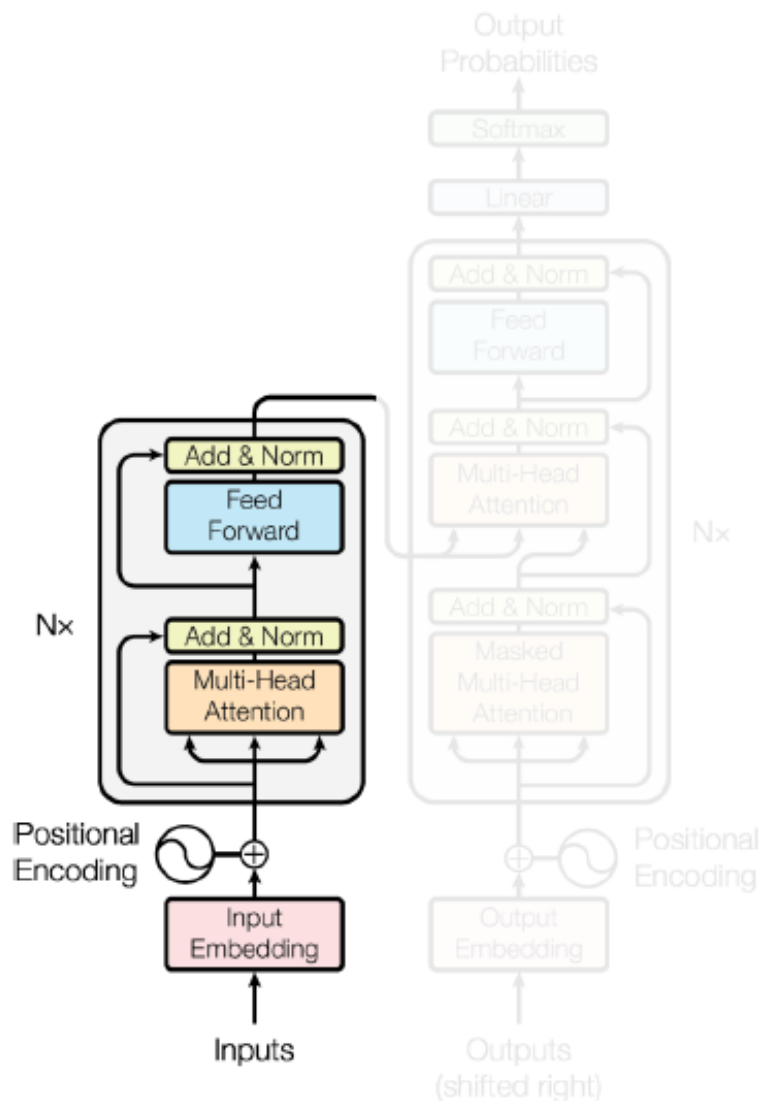
Only use the left side of the architecture, aka the encoder

The encoder takes raw text and encodes it as an embedding representation of the text

In short, ***it understands text***

Application: Sentiment Analysis

- **Input:** "I love cold lemonade!"
- **Output:** Positive



Decoder Only Model

Decoder-Only Models

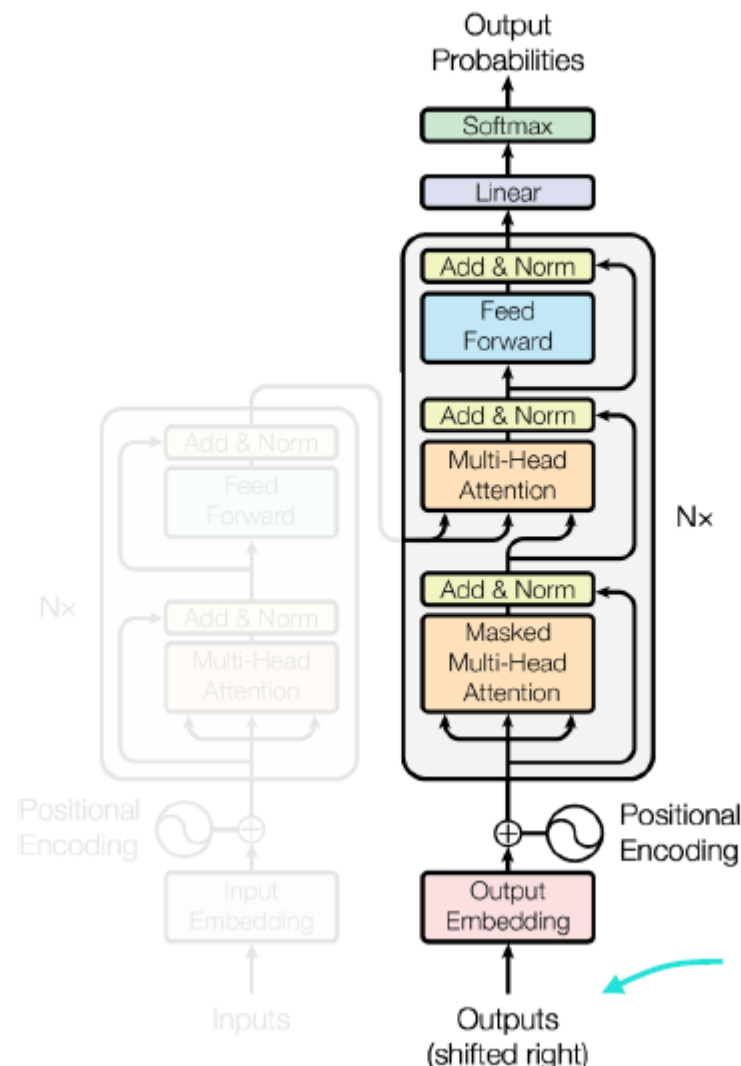
Only use the right side of the architecture, aka the decoder

The decoder takes an input text sequence and infers* the next word

In short, **it generates text**

Application: Text Generation

- **Input:** "I love cold lemonade!"
- **Output:** "It's the perfect drink! 🍋 🥤"



Encoder – Decoder Model

Encoder-Decoder Models

Use the entire architecture, both the encoder and decoder sides

The encoder-decoder takes two inputs:

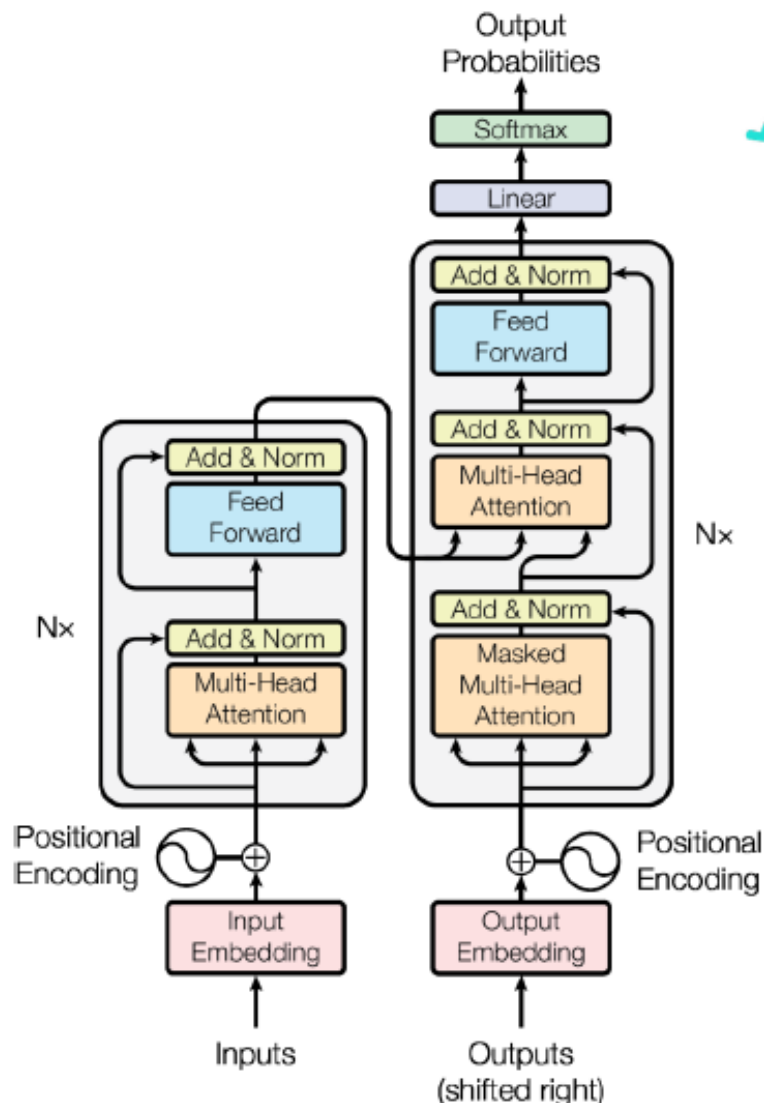
1. A text sequence
2. A shifted target sequence

Both are encoded as embeddings and combined to infer the next word

In short, *it understands and generates text*

Application: Translation

- **Input:** "I love cold lemonade!"
- **Output:** "¡Me encanta la limonada fría!"



Summary

Encoder-Only LLMs

Turns text into embeddings

Popular models:

- BERT – Bidirectional Encoder Representations from Transformers



A base LLM can have many variants:

- *RoBERTa – better performance*
- *DistilBERT – smaller and faster*
- *BERT-QA – fine-tuned for question answering*
- *BioBERT – fine-tuned for biomedical texts*
- *LegalBERT – fine-tuned for legal texts*

Decoder-Only LLMs

Infers the next token in a text

Popular models:

- GPT – Generative Pre-trained Transformer

Encoder-Decoder LLMs

Turns text into other text

Popular models:

- T5 – Text-to-Text Transfer Transformer
- BART – combines BERT and GPT



Once trained, **all the parameters are set** (*embeddings, attention weights, FNN weights, etc.*) so that anyone can input in a new text sequence and quickly calculate an output

Summary

LLM	Type	Created By	Parameters	Trained On
BERT	Encoder-only	Google	110 million (BERT-base) 340 million (BERT-large)	BooksCorpus (800M words) English Wikipedia (2.5 billion words)
DistilBERT	Encoder-only	Hugging Face	66 million	BooksCorpus (800M words) English Wikipedia (2.5 billion words)
T5	Encoder-Decoder	Google	220 million (T5-small) 770 million (T5-large) 11 billion (T5-11B)	C4 (Colossal Clean Crawled Corpus) ~750GB of text
BART	Encoder-Decoder	Meta (Facebook)	140 million (BART-base) 400 million (BART-large)	CC-News (140GB of news articles) BooksCorpus (800M words) English Wikipedia (2.5 billion words)
GPT	Decoder-only	OpenAI	175 billion (GPT-3) 1.5 billion (GPT-2)	WebText (40GB of text from Reddit links) BooksCorpus (800M words) English Wikipedia (2.5 billion words)

Many of these are open source, but GPT-3 and later versions are proprietary – OpenAI hasn't even shared the number of parameters for GPT-4!



Each LLM can be used for **multiple applications**:

- BERT is used for sentiment analysis, classification, etc.
- T5 is used for summarization, translation, etc.
- GPT is mainly used for generation
- Any of them can be used for question answering, etc.

Working with Transformers: Workflow

- 1 **Determine your goal** (*Sentiment analysis, summarization, generation, etc.*)
- 2 **Identify a pretrained model** from Hugging Face's Model Hub
- 3 **Specify your input data** (*a single string, a Series or column of text data, etc.*)
- 4 **Apply the pretrained model** on your input data and view the outputs

Encoder-Only

- Sentiment Analysis
- Named Entity Recognition

Decoder-Only

- Text Generation

Encoder-Decoder

- Zero-Shot Classification
- Text Summarization

Embedding

- Document Similarity

Sentiment Analysis

Sentiment analysis is used to determine the positivity or negativity of text

- The default LLM for sentiment analysis is **DistilBERT** (*encoder-only*)

```
from transformers import pipeline

sentiment_analyzer = pipeline("sentiment-analysis",
                              model="distilbert/distilbert-base-uncased-finetuned-sst-2-english",
                              device=-1)

text = 'When life gives you lemons, make lemonade! 😊'

sentiment_analyzer(text)
```

```
[{'label': 'POSITIVE', 'score': 0.996239423751831}]
```

The predicted
sentiment is **positive**
(vs. negative)

This is the model's
confidence in its prediction
(from 0 to 1)

Transformer pipeline steps:

1. Import the pipeline module
2. Specify the task: **sentiment-analysis**
3. Choose the default model
4. Specify we're only using our CPU

Sentiment Analysis

Sentiment analysis is used to determine the positivity or negativity of text

- The default LLM for sentiment analysis is **DistilBERT** (*encoder-only*)

```
from transformers import pipeline

sentiment_analyzer = pipeline("sentiment-analysis",
                               model="distilbert/distilbert-base-uncased-finetuned-sst-2-english",
                               device=-1)

text = 'When life gives you lemons, make lemonade! 😊'

sentiment_analyzer(text)
```

```
[{'label': 'POSITIVE', 'score': 0.996239423751831}]
```

The predicted
sentiment is **positive**
(vs. negative)

This is the model's
confidence in its prediction
(from 0 to 1)

Transformer pipeline steps:

1. Import the pipeline module
2. Specify the task: **sentiment-analysis**
3. Choose the default model
4. Specify we're only using our CPU

Named Entity Recognition (NER)

Named Entity Recognition (NER) is used to find and label important information (*people, places, organizations, dates, etc.*) in text

- The default LLM for NER is **BERT** (*encoder-only*)

```
from transformers import pipeline

ner_analyzer = pipeline("ner",
                        model='dbmdz/bert-large-cased-finetuned-conll03-english',
                        device=-1,
                        aggregation_strategy='SIMPLE')

text = "I ordered an Arnold Palmer at Applebee's in Springfield."

ner_analyzer(text)
```

Transformer pipeline steps:

1. Import the pipeline module
2. Specify the task: **ner**
3. Choose the default model
4. Specify we're only using our CPU

```
[{'entity_group': 'MISC',
  'score': np.float32(0.9914088),
  'word': 'Arnold Palmer',
  'start': 13,
  'end': 26},
 {'entity_group': 'ORG',
  'score': np.float32(0.9436141),
  'word': "Applebee ' s",
  'start': 30,
  'end': 40},
 {'entity_group': 'LOC',
  'score': np.float32(0.9780036),
  'word': 'Springfield',
  'start': 44,
  'end': 55}]
```

By setting `aggregation_strategy` to "SIMPLE", we're specifying we want to look at words, not subwords



We're 97.8% sure "Springfield" is a named entity identified as a location (LOC) – the word starts at character 44 and ends at 55

NER Applications

- Information Extraction: Extract names, dates, and locations to create concise summaries.
- Question Answering Systems: Answer entity-specific questions: Helps systems identify the entity being asked about
- Finance and Business Intelligence: Automated data extraction from reports: Identify companies, stock tickers, currencies, and dates.
- Social Media Monitoring Trend analysis: Detect entities being discussed online (brands, celebrities, events).

Zero-Shot Classification

- Categorizes text without training with labels
- We use an encoder-decoder model (BART) for this setup

```
from transformers import pipeline

classifier = pipeline("zero-shot-classification",
                      model="facebook/bart-large-mnli",
                      device=-1)

text = 'When life gives you lemons, make lemonade! 😊'

classifier(text, ['quote', 'food & drinks', 'technology'])

{'sequence': 'When life gives you lemons, make lemonade! 😊',
 'labels': ['quote', 'food & drinks', 'technology'],
 'scores': [0.9833195209503174, 0.011176466010510921, 0.005504013504832983]}
```

Same pipeline steps with **zero-shot-classification** as the task



This is a quote!



You provide the label options and the model returns scores that classify it into one of those labels (adding up to 1)

Text Summarization

- Condense a large body of text into a meaningful summary

```
from transformers import pipeline
```

```
summarizer = pipeline("summarization",  
                      model="facebook/bart-large-cnn",  
                      device=-1)
```

← Same pipeline steps with
summarization as the task

```
text = """The lemon tree produces a pointed oval yellow fruit. Botanically this is a hesperidium, a modified berry with a tough, leathery rind. The rind is divided into an outer colored layer or zest, which is aromatic with essential oils, and an inner layer of white spongy pith. Inside are multiple carpels arranged as radial segments. The seeds develop inside the carpels. The space inside each segment is a locule filled with juice vesicles. Lemons contain many phytochemicals, including polyphenols, terpenes, and tannins.[3] Their juice contains slightly more citric acid than lime juice (about 47 g/L), nearly twice as much as grapefruit juice, and about five times as much as orange juice.[4]"""
```

```
summarizer(text, min_length=20, max_length=50)
```

```
'The lemon tree produces a pointed oval yellow fruit. Lemons contain many phytochemicals, including polyphenols, terpenes, and tannins. Their juice contains slightly more citric acid than lime juice.'
```



Beyond specifying the min and max length of the summarized text, you can **set the `do_sample` parameter** to False to use the most likely next word (default) or to True to use a more random and creative next word

Text Generation

- Generate new text based on a prompt

```
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="gpt2", device=-1)
```

```
prompt = "On a hot summer day, I love to drink cold lemonade because"
```

```
generator(prompt, max_length=50, num_return_sequences=1, do_sample=False)
```

```
[{'generated_text': "On a hot summer day, I love to drink cold lemonade because it's so refreshing. I also love to drink cold lemonade because it's so refreshing.\n\nI love to drink cold lemonade because it's so refreshing. I also"}]
```

```
generator(prompt, max_length=50, num_return_sequences=1, do_sample=True)
```

```
[{'generated_text': "On a hot summer day, I love to drink cold lemonade because there really are no other options. There's no way it'll keep me warm and dry, if it does end up freezing, it won't be good for you. It's"}]
```

```
generator(prompt, max_length=50, num_return_sequences=1, do_sample=True)
```

```
[{'generated_text': "On a hot summer day, I love to drink cold lemonade because I am always having a good time doing it! I've found it is easier to drink lemonade without actually making any lemonade.\n\nThis hot summer day I was inspired"}]
```

Text Generation

- Generate new text based on a prompt

```
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="gpt2", device=-1)
```

```
prompt = "On a hot summer day, I love to drink cold lemonade because"
```

```
generator(prompt, max_length=50, num_return_sequences=1, do_sample=False)
```

```
[{'generated_text': "On a hot summer day, I love to drink cold lemonade because it's so refreshing. I also love to drink cold lemonade because it's so refreshing.\n\nI love to drink cold lemonade because it's so refreshing. I also"}]
```

```
generator(prompt, max_length=50, num_return_sequences=1, do_sample=True)
```

```
[{'generated_text': "On a hot summer day, I love to drink cold lemonade because there really are no other options. There's no way it'll keep me warm and dry, if it does end up freezing, it won't be good for you. It's"}]
```

```
generator(prompt, max_length=50, num_return_sequences=1, do_sample=True)
```

```
[{'generated_text': "On a hot summer day, I love to drink cold lemonade because I am always having a good time doing it! I've found it is easier to drink lemonade without actually making any lemonade.\n\nThis hot summer day I was inspired"}]
```

Document Embeddings

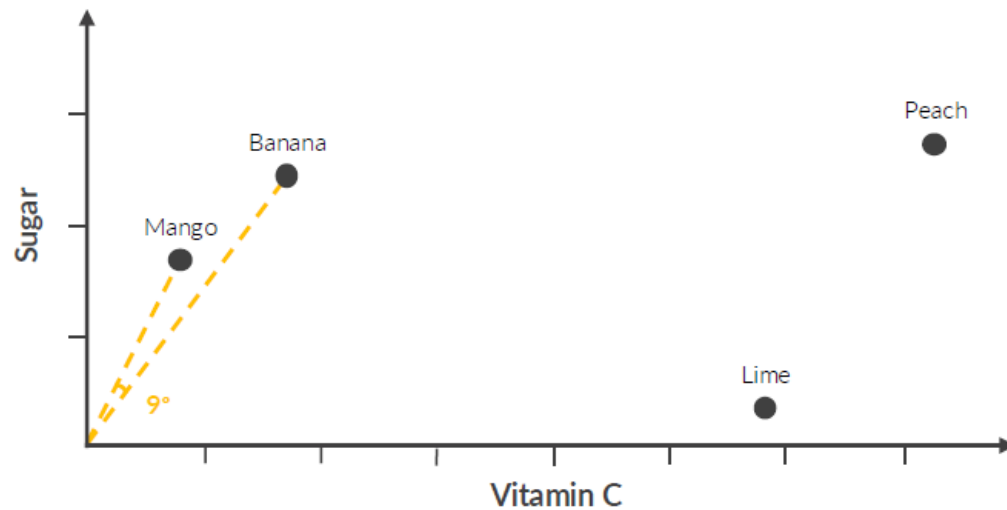
- Represents document as series of numbers: provides a mathematical representation.
- We use encoders only model
- The embeddings or vector representations are used for downstream tasks such as: classification, clustering, information retrieval

Cosine Similarity

- Values range from -1 (dissimilar) to 1 (similar)

EXAMPLE

Finding the fruit most similar to mango



$$\cos(9) = 0.98$$



Bananas are the most similar fruit to mangos!

Review

These are the **NLP techniques & applications** that we covered:

NLP Category	Technique	Application
 Traditional	Rules-Based	→ Sentiment Analysis
	Supervised Learning (Naïve Bayes)	→ Text Classification
	Unsupervised Learning (NMF)	→ Topic Modeling
 Modern	Encoder-Only LLM (BERT)	→ Sentiment Analysis Named Entity Recognition (NER)
	Encoder-Decoder LLM (BART)	→ Zero Shot Classification Text Summarization
	Decoder-Only LLM (GPT)	→ Text Generation
	Embeddings (MiniLM)	→ Document Similarity